G1 Digital Twin — Simulation Test Instructions

Rev. 2026-08-07 · Branch **tutor-feedback-metareasoner-sim** · Meta state machine (supervisor's spec, batches 1+2) + calibrated sensor noise + synthetic camera (DOOR-VIS in the twin) · All tests run on the Mac against the g1sim Docker container. Source: `docs/src/`, rebuild with `tools/build_docs.sh`.

0 Prerequisites (once per session)

1. Check the container and simulator are alive:

   ```
   docker ps | grep g1sim
   ```

   If `g1sim` is not running, start it. If the container was **restarted**, Gazebo and rosbridge must be relaunched (rosbridge is always manual):

   ```
   docker exec -d g1sim bash -c "source /opt/ros/humble/setup.bash && source /home/ubunt
   u/g1_ws/install/setup.bash && ros2 launch g1_sim lab.launch.py gui:=false > /tmp/lab_l
   aunch.log 2>&1"
   ```

   ```
   docker exec -d g1sim bash -c "source /opt/ros/humble/setup.bash && source /home/ubunt
   u/g1_ws/install/setup.bash && ros2 launch rosbridge_server rosbridge_websocket_launch.
   xml port:=8765"
   ```

2. If port 6080 (noVNC) is taken by the auto-started `ros2_humble` container: `docker stop ros2_humble`.
3. Check out the branch:

   ```
   cd "/Users/adrianlendinezibanez/Claude/Projects/G1 ROBOT" && git checkout tutor-feedba
   ck-metareasoner-sim
   ```

Full rebuild instructions for the twin (Dockerfile, workspace, noVNC access and image backup) are in `sim/RECONSTRUIR_Y_LANZAR.md`.

1 Flag reference

Flag	Default	Meaning
`G1_METASM`	1	Meta state machine: NORMAL / DEGRADED (cap 0.24) / BLIND (cap 0.28) / RECOVERY / ASSIST (stop & wait for human `m <cm>`)
`G1_RETREAT`	1	Retreat v2: breadcrumb reverse on confirmed stuck, max 3/run, pauses the P9 HELP clock
`G1_EXIT_RETREAT`	1	Retreat-at-exit: near the start point the span guard drops 0.5 → 0.15 m
`G1_SIM_NOISE`	0	Calibrated sensor noise (adapter only): per-ray σ 0.09 m + AR(1) bias + Poisson bursts + odom drift 0.07 m/m, 1.5°/m
`G1_SIM_PERC`	0	Synthetic camera: perception service on 127.0.0.1:8010 (door detection from true sim pose; latency 0.28 s, dropout 4%, visible <2.8 m and ±28°)
`G1_PERC`	—	Set to `127.0.0.1:8010` when using `G1_SIM_PERC=1`
`G1_DOOR_VIS`	0	Door heading from vision (the golden-window mechanism); needs the camera

Every run below also uses the standard set: `G1_META2=2 G1_M2_CFG=config_meta2_g1payload.json G1_M2_L3=1 G1_M2_L4=1 G1_M2_PROFILES=1 G1_M2_DOORLIB=1 G1_M2_FRAGSPEED=1 G1_M2_ABORT_WIN=150 G1_M2_ABORT_PROG=0.25 G1_M2_HELP_S=16` plus a per-experiment `G1_SIM_ID` and a **fresh** `G1_M2_STATE` (delete

the file before a new experiment; never reuse another experiment's state). Configs and state are passed as bare filenames and resolved from `config/` and `state/`.

2 Test 1 — Clean baseline, machine ON (A→B)

```
docker exec g1sim bash -c "gz model -m g1 -x 0.99 -y 0.57 -z 0.05 -Y -2.1"
```

```
cd "/Users/adrianlendinezibanez/Claude/Projects/G1 ROBOT" && G1_META2=2 G1_M2_CFG=config_meta2_g1payload.json G1_SIM_ID=lab_v1_msm G1_M2_STATE=meta2_state_msm_test.json G1_M2_L3=1 G1_M2_L4=1 G1_M2_PROFILES=1 G1_M2_DOORLIB=1 G1_M2_FRAGSPEED=1 G1_M2_ABORT_WIN=150 G1_M2_ABORT_PROG=0.25 G1_M2_HELP_S=16 /Users/adrianlendinezibanez/unitree_webrtc_connect/.venv/bin/python g1_sim_adapter.py goto B
```

Analyse the newest dataset:

```
python3 analysis/analyze_msm.py "$(ls -t dataset/2026*_ours_B.json | head -1)"
```

Accept if: **REACHED in 95–115 s, 0 collisions, 0 retreats**; meta states ≈ NORMAL 55–60% / BLIND 40–45% with transitions at the furniture band (x≈−1.8), the door (−2.6…−3.9) and the B approach (−4.3…−4.8). Reference runs: 106.0 s and 95.5 s (30-jul), 94 s and 89 s (07-aug, after the repo reorganisation).

3 Test 2 — Human support channel (collision reports + DEGRADED)

Start a clean A→B run as in Test 1. While it runs, report three collisions the robot did not detect (~4 s apart, from a second terminal). Either press `c` three times in `spill_mark.py`, or send the datagrams:

```
python3 -c "import socket,time; s=socket.socket(socket.AF_INET,socket.SOCK_DGRAM); [ (s.sendto(b'hcol',('127.0.0.1',7777)), time.sleep(4)) for _ in range(3) ]"
```

Accept if: the log shows `COLISION reportada` ×3 and `laser_lied` ×3; `laser_trust` drops to ≈0.30; one RETREAT fires "por colision" (2 reports <20 s = confirmed trouble); after the retreat the machine shows **DEGRADED** (cap 0.24) briefly and then recovers. Reference run: 20260730_185420.

4 Test 3 — Recovery → human assistance (trap at the door)

1. Teleport to B:

   ```
   docker exec g1sim bash -c "gz model -m g1 -x -4.75 -y 2.60 -z 0.05 -Y -0.9"
   ```

2. Spawn the trap box at the door mouth:

   ```
   docker exec g1sim bash -c "source /opt/ros/humble/setup.bash && ros2 service call /spawn_entity gazebo_msgs/srv/SpawnEntity \"{name: 'trampa_msm', xml: '<sdf version=\\\"1.6\\\"><model name=\\\"trampa_msm\\\"><static>true</static><link name=\\\"l\\\"><collision name=\\\"c\\\"><geometry><box><size>0.45 0.45 0.5</size></box></geometry></collision><visual name=\\\"v\\\"><geometry><box><size>0.45 0.45 0.5</size></box></geometry></visual></link></model></sdf>', initial_pose: {position: {x: -3.30, y: 0.60, z: 0.25}}}\""
   ```

3. Run `goto A` (Test 1 command with `goto A`). The robot wedges in the door approach, retreats up to 3 times, then **stops in ASSIST** printing `ASISTENCIA: parado esperando al humano`.
4. Act as the human: delete the box, then send the assistance (or press `m 40` in the marker):

```
docker exec g1sim bash -c "source /opt/ros/humble/setup.bash && ros2 service call /del
ete_entity gazebo_msgs/srv/DeleteEntity \"{name: 'trampa_msm'}\""
```

```
python3 -c "import socket; socket.socket(socket.AF_INET,socket.SOCK_DGRAM).sendto(b'hm
ove:40',('127.0.0.1',7777))"
```

Accept if: goto.log shows `asistencia RECIBIDA → presupuestos re-armados`, the retreat budget restarts at n=1/3, and the robot resumes. The dataset must contain `human_assist` events and meta_state transitions RECOVERY→ASSIST→BLIND. If nobody helps, the P9 safety net must close the run. Reference run: 20260730_184256 (two assists consumed in one run).

5 Test 4 — Closed-loop A/B campaigns (the headline result)

Each campaign is 14 interleaved B→A runs (6 OFF / 6 ON / 2 ON+simulated human) and takes ~40–70 min. Results append to a JSON in `results/`.

```
python3 -u campaigns/sim_ab_msm_campaign.py          # base (no noise)
```

```
python3 -u campaigns/sim_ab_msm_noise_campaign.py    # under calibrated noise
```

Accept if (reference 31-jul):

	OFF (golden)	ON (machine)
Base	6/6 reached, median ≈93 s	6/6 reached, ≈101 s (+8–10%), **0 false interventions**
Calibrated noise	≈3/6 reached, ~27 collisions (door + B-pocket grind)	**6/6 reached, 0 collisions (prevention via BLIND caps)**

The noise outcome is stochastic: expect 2–4 OFF failures per 6, always at the door mouth (–3.3, 0.6) or the B-pocket (–4.5, 2.1). ON may occasionally draw a pocket wedge too — then retreats + ASSIST must engage (that is a pass, not a fail).

6 Test 5 — Full golden config in the twin (noise + camera + DOOR-VIS)

```
docker exec g1sim bash -c "gz model -m g1 -x -4.75 -y 2.60 -z 0.05 -Y -0.9"
```

```
cd "/Users/adrianlendinezibanez/Claude/Projects/G1 ROBOT" && G1_SIM_NOISE=1 G1_SIM_PERC=1 G1
_PERC=127.0.0.1:8010 G1_DOOR_VIS=1 G1_META2=2 G1_M2_CFG=config_meta2_g1payload.json G1_SIM_I
D=lab_v1_spb G1_M2_STATE=meta2_state_spb.json G1_M2_L3=1 G1_M2_L4=1 G1_M2_PROFILES=1 G1_M2_D
OORLIB=1 G1_M2_FRAGSPEED=1 G1_M2_ABORT_WIN=150 G1_M2_ABORT_PROG=0.25 G1_M2_HELP_S=16 /Users/
adrianlendinezibanez/unitree_webrtc_connect/.venv/bin/python g1_sim_adapter.py goto A
```

Accept if: startup prints `CAMARA SINTETICA ON` and `[perc] … OK`; **REACHED ≈100–110 s, 0 collisions**; goto.log shows `DOOR-VIS activo`; the dataset has `door_b` samples only near the door (~150–170) and a `door_crossed` event at ≈35–45 s. Reference runs: 20260731_105215 / 105402 (both 102 s).

7 Shadow replay over past data (no simulator needed)

Replays the state machine over any recorded dataset (real or twin) and reports occupancy, would-be retreat triggers and help requests:

```
python3 analysis/replay_msm.py dataset/20260724_163736_ours_A.json
```

Accept if: golden runs → no triggers; the collapse-night failures → triggers + assist requests. Reference over 309 real runs: 2/192 false positives on clean runs, action in 92% of failed runs, `laser_lied` on 55% of real collisions.

8 Method rules (they cost sessions to learn)

- **One change per batch**; never edit code while a campaign is running (each run re-reads the sources).
- Manual runs bypass the campaign reset — **always teleport first**; delete the experiment's state file to start fresh.
- Instrumentation down (marker/camera server) = run invalid, repeat it.
- Noise results live in their own baseline lane — never compare noisy times against no-noise times.
- Kill stray processes by PID, never `pkill -f`.
- goto.log rotation: before pushing from the Mac, GPUEDGE must commit+push its own goto.log appends first, then pull.

Git note. The branch `tutor-feedback-metareasoner-sim` is the published copy of this work. The original `tutor-feedback-metareasoner` ref on GitHub is intentionally left behind until the goto.log coordination step with GPUEDGE is done.